CONDUCTOR OS

Personal Intelligence Operating System

A New Paradigm for Human–Digital Interaction

White Paper Version 1.0 March 2026
Architecture, Stack, and Implementation Roadmap

ABSTRACT

Conductor OS is a full-stack personal intelligence layer that mediates between a human operator and the entirety of their digital ecosystem. Unlike conventional dashboards that aggregate and display information, Conductor OS treats the human as a conductor — an active director whose current intentions reshape the priority, routing, and transformation of every signal from every connected service. The system is built on a tiered AI architecture: lightweight local models run continuously at zero cost; free-tier inference APIs handle semantic enrichment asynchronously; and large language models are invoked on-demand for synthesis and strategic reasoning. The result is a living, adaptive interface that learns the operator's context, surfaces non-obvious connections, and enables bidirectional action across all connected platforms from a single unified workspace.

Table of Contents

1. The Problem — The Fragmented Digital Self3
2. Core Paradigm — The Conductor Model4
3. System Architecture Overview5
4. The Five Processing Layers6
5. AI Stack — Local, Free API, LLM Gateway8
6. Full Technology Stack10
7. Integration Catalog12
8. Data Architecture14
9. Intention Engine — The Core Differentiator15
10. Security and Privacy Model16
11. Deployment Architecture17
12. Implementation Roadmap18
13. Use Cases19
14. Open-Source Model and Community21
15. Appendix — API Reference22

1. The Problem — The Fragmented Digital Self

The modern knowledge worker, creator, or entrepreneur navigates an average of 14 to 22 separate digital services each day. Email clients, project management tools, communication platforms, financial trackers, content feeds, code repositories, and social networks each operate as isolated silos — each demanding context-switching, each designed to maximize engagement with itself rather than to serve the operator's actual goals.

This fragmentation creates three systemic pathologies:

- Context-switching tax. Every transition between tools costs 20 to 30 minutes of refocus time (per Gloria Mark, UC Irvine). An operator managing 10 services loses 3 to 5 hours of deep work per day not to the tasks themselves but to navigation overhead.
- Passive consumption bias. Every existing aggregator — RSS readers, social feeds, news apps — is built for consumption, not direction. The operator is a receiver, not a conductor. Algorithms optimize for engagement, not for the operator's stated goals.
- Action friction. Insight without frictionless action is noise. Reading an important article, discovering a key connection, or noticing a critical signal all require manual, multi-step workflows to translate into action — open another app, copy text, find the right recipient, remember the context.

> *The root cause is architectural: every tool is designed as a destination. Conductor OS is designed as an instrument.*

Existing solutions fail in predictable ways. Zapier and n8n solve integration but not intelligence. Notion AI solves knowledge management but not real-time signal processing. News aggregators solve consumption but not action. LLM chatbots solve synthesis but operate in isolation from live data. No system to date has unified the full loop: signal ingestion → semantic processing → intention-aware filtering → LLM synthesis → bidirectional action.

2. Core Paradigm — The Conductor Model

The central design insight of Conductor OS is the inversion of the operator's role from passive consumer to active director. In music, a conductor does not produce sound — the orchestra does. The conductor's function is to impose intention on raw potential: to shape timing, emphasis, and coherence from many independent instruments into a unified performance.

Conductor OS applies this metaphor architecturally. Every connected service — Telegram, GitHub, YouTube, financial APIs, email, RSS feeds — is an instrument in the operator's digital orchestra. Each produces signal continuously. The Conductor OS layer mediates between these signals and the operator, applying intention-aware processing that transforms noise into meaning.

2.1 The Three Laws of the Conductor Model

Law 1 — Intention precedes data

The system does not aggregate and then present. It first asks: what is the operator's current goal? All subsequent processing is shaped by the answer. An operator in "deep research" mode receives different signal prioritization, different LLM prompting, and different action affordances than an operator in "monitoring" or "creative synthesis" mode.

Law 2 — Bidirectionality is not optional

Every inbound signal surface must have a corresponding outbound action pathway. Reading a GitHub issue should offer a one-gesture path to create a Notion task, reply in Telegram, or trigger an n8n workflow. The interface is not a viewer — it is a control surface.

Law 3 — Intelligence must be proportional

Not every signal requires a large language model. Routine classification, deduplication, and relevance scoring are handled by local models running in milliseconds at zero marginal cost. LLMs are reserved for synthesis, strategic reasoning, and tasks where reasoning quality justifies latency and cost. This tiering is what makes the system economically viable as a personal tool.

2.2 Information Entropy as a Design Parameter

Conductor OS treats information entropy not as a problem to eliminate but as a system parameter to tune. A filter optimized purely for relevance creates a closed attractor — the operator receives

only what confirms existing priorities. Research in complex systems shows that cognitive flexibility requires sustained exposure to adjacent-domain signals.

The Wildcard Channel implements this principle formally. At default settings, 10 percent of the processed feed is drawn from outside the operator's stated interest graph — philosophy when the session is focused on DeFi; systems theory when the session is focused on film production. This parameter is adjustable from 0 percent (pure focus) to 40 percent (maximum serendipity) and is tracked over time to measure its effect on synthesis quality.

3. System Architecture Overview

Conductor OS is structured as a layered system with clear separation between signal acquisition, semantic processing, intention-aware filtering, action dispatch, and LLM synthesis. The architecture is designed to be deployed as a self-hosted stack, with each layer independently replaceable.

Architecture at a Glance

```
Layer 5  (Synthesis)      Claude Sonnet API — on-demand LLM synthesis,
digest, strategic Q&A
Layer 4  (Action)         Dispatch engine — n8n workflows, Telegram sends,
GitHub operations
Layer 3  (Intention)      Adaptive filter — goal-weighted scoring, wildcard
injection, persona TF-IDF
Layer 2  (Semantic)       BERT-NER, DistilBERT sentiment — HuggingFace
Inference API, async batch
Layer 1  (Topic)          LDA classification, TF-IDF cosine — local process,
~3ms, zero cost
Layer 0  (Ingestion)      Universal Connector — RSS, Telegram webhooks,
GitHub Events, REST adapters
```

The architecture follows a strict principle of compute proportionality. Layers 0 through 1 run as persistent background processes — always on, zero marginal cost, sub-10ms latency. Layer 2 runs as an asynchronous enrichment queue — never blocking the UI, processing items in the background. Layer 3 runs on every UI render cycle in under 1ms. Layer 4 executes on explicit operator action. Layer 5 executes on explicit operator request.

4. The Five Processing Layers

4.1 Layer 0 — Universal Ingestion

Layer 0 is the normalization boundary. Every incoming signal, regardless of source format, is transformed into a canonical OmniSignal object before any further processing. This design means that adding a new data source requires only a new adapter — the rest of the pipeline is source-agnostic.

```
interface OmniSignal {
  id:          string          // deterministic hash of source + content
  source:      string          // adapter identifier: "telegram", "github", "rss"
  sourceType:  SignalType      // STREAM | WEBHOOK | POLL | PUSH
  operator:    string          // persona ID
  content:     string          // normalized UTF-8 text body
  title?:      string          // extracted title if available
  url?:        string          // canonical source URL
  author?:     string          // sender/author if available
  timestamp:   Date            // UTC, network-jitter compensated
  raw:         unknown         // original source payload (preserved)
  meta:        Record<string, unknown>
}
```

4.2 Layer 1 — Local Semantic Processing

Layer 1 runs entirely within the Node.js process — no network calls, no external services. It performs three operations on every OmniSignal:

- Topic classification via lightweight LDA simulation using pre-computed keyword co-occurrence matrices. Output: topic vector with confidence scores across up to 12 operator-defined topic domains.
- Relevance scoring via TF-IDF cosine similarity against the operator's current intention vector. The intention vector is recomputed whenever the operator changes goal mode. Output: relevance score 0.0–1.0.
- Entity pre-extraction via regex pattern matching against a curated entity registry (cryptocurrency tickers, technology names, platform names, known organizations). Output: entity mention list for enrichment in Layer 2.

4.3 Layer 2 — Asynchronous Semantic Enrichment

Layer 2 sends signals to external inference endpoints in background batches. It never blocks the UI. Items are processed in order of relevance score — high-relevance items are enriched first. Two models are called via the HuggingFace Inference API free tier:

- dslim/bert-base-NER for named entity recognition. Identifies persons, organizations, locations, and domain-specific technical entities. Average latency: 180–250ms. Free tier quota: sufficient for approximately 30,000 items per month.
- distilbert-base-uncased-finetuned-sst-2-english for sentiment classification. Returns POSITIVE/NEGATIVE with confidence. Average latency: 120–200ms. Local dictionary fallback activates automatically when quota is exhausted.

A local Ollama instance can replace the HuggingFace endpoints entirely for operators who require full offline operation or have privacy requirements that preclude external API calls.

4.4 Layer 3 — Adaptive Intention Filter

Layer 3 is the core differentiator of the Conductor OS paradigm. It runs on every feed render cycle and applies the operator's current intention state to re-rank and re-filter the processed signal pool. The filter has three components:

- Goal-mode weight matrix. Each goal mode (RESEARCH, MONITORING, CREATIVE, EXECUTION, COMMUNICATION) carries a weight vector that multiplies against each signal's topic scores. A signal about ZK cryptography scores 3x higher in RESEARCH mode than in COMMUNICATION mode.
- Recency decay. Signals older than the operator's configured attention window are progressively downweighted. Default window: 6 hours. Configurable per source and per topic.
- Wildcard injection. A configurable percentage of feed slots are reserved for signals outside the operator's current intention vector, drawn from adjacent topic domains. Default: 10%. Prevents closed attractor formation.

4.5 Layer 4 — Bidirectional Action Dispatch

Layer 4 converts intention into action. Every signal card in the UI exposes a set of context-aware action affordances — the available actions depend on the signal's source, the operator's connected services, and the current goal mode. Actions are dispatched through n8n workflow triggers, which handle authentication, retry logic, and delivery confirmation.

Example action pathways: save to Notion research database; forward to Telegram channel with AI-generated annotation; create GitHub issue; add to content calendar; trigger custom n8n workflow; archive with topic tags.

5. AI Stack — Local, Free API, LLM Gateway

The AI architecture of Conductor OS is designed around the principle of compute proportionality: use the cheapest sufficient model for each task. This is not a compromise — it produces better outcomes than routing all tasks through a single large model, because small specialized models are faster, more consistent, and more predictable for classification tasks.

5.1 Tier 1 — Local Models (Zero Cost, Sub-10ms)

Component	Specification
Topic classifier	Keyword LDA — 12 topic domains, configurable vocabulary, pure JavaScript, no dependencies
Relevance scorer	TF-IDF cosine similarity — operator interest vector updated on intention change
Entity pre-filter	Regex pattern matching against 2,400-entry entity registry, extensible
Deduplication	SHA-256 content hash with Bloom filter — eliminates duplicate signals across sources
Sentiment fallback	AFINN-165 dictionary scorer — activates when HF quota is exhausted
Runtime	Node.js process, persistent, shares memory with main application server

5.2 Tier 2 — HuggingFace Inference API (Free Tier)

The HuggingFace Inference API provides access to state-of-the-art transformer models without infrastructure overhead. The free tier is sufficient for personal use. The system implements intelligent quota management: high-relevance signals are enriched first; a local fallback activates transparently when limits are reached.

Model	Task	Notes
dslim/bert-base-NER	Named entity recognition	Persons, orgs, locations, tech entities
distilbert-sst-2	Sentiment analysis	POSITIVE/NEGATIVE + confidence
facebook/bart-large-cnn	Extractive summarization	Optional — for long-form signals
sentence-transformers/all-MiniLM-L6-v2	Semantic similarity	Cross-source duplicate detection

5.3 Tier 3 — Local LLM via Ollama (Optional, Privacy Mode)

Operators who require full data sovereignty can replace Tier 2 and Tier 3 with a locally-running Ollama instance. Recommended models by task:

- Classification and NER: llama3.2:3b — fast, sufficient accuracy for structured tasks
- Synthesis and Q&A: qwen2.5:14b or llama3.1:8b — good balance of quality and speed on consumer hardware
- Summarization: mistral:7b — strong extractive summarization performance

Ollama integration is plug-and-play via the Vercel AI SDK's Ollama provider. Switching between cloud and local inference requires a single environment variable change.

5.4 Tier 4 — Claude LLM Gateway (On-Demand)

The Claude Sonnet API is the system's highest-capability reasoning layer, called only when the operator explicitly requests synthesis, strategic analysis, or deep Q&A. It is never called for routine processing. This constraint keeps operational costs under $0.05 per day for typical usage.

All Claude API calls are injected with the operator's current persona context — name, role, active intention mode, interest graph, and a structured summary of recent feed signals. This produces responses that are specific to the operator's situation rather than generic.

```
// Persona injection pattern
const systemPrompt = `
You are the intelligence layer for ${persona.name}, ${persona.role}.
Current intention mode: ${persona.activeGoal}.
Active interest domains: ${persona.interests.join(", ")}.
Context window: last ${contextItems.length} signals from ${sources.join(", ")}.

Your role: surface non-obvious connections, provide strategic insight,
and recommend specific actions. Be incisive, not general.
`;
```

6. Full Technology Stack

Layer	Technology	Rationale
Frontend framework	Next.js 15 (App Router)	Server components, streaming UI, edge-compatible
UI components	shadcn/ui + Tailwind CSS	Accessible, fully customizable, not a black box
State management	Zustand + React Query	Lightweight global state + server sync
Real-time transport	Supabase Realtime (WebSocket)	Managed, scales to thousands of concurrent signals
Database	PostgreSQL via Supabase	pgvector for semantic search, Row Level Security
Auth	Supabase Auth	OAuth (GitHub, Google), magic link, JWT
File storage	Supabase Storage	Signal attachments, persona configs, exports
Background jobs	BullMQ + Redis (Upstash)	Async enrichment queue, reliable delivery, retries
Workflow automation	n8n (self-hosted)	Action dispatch, webhook ingestion, integrations
LLM inference	Vercel AI SDK	Unified interface for Claude, Ollama, OpenAI
Local AI	Ollama	Privacy mode — Llama 3.2, Qwen 2.5, Mistral
Free AI tier	HuggingFace Inference API	NER, sentiment, similarity — no key required
API layer	tRPC	End-to-end type safety, auto-generated client
Deployment	Vercel (frontend) + Railway/Render (backend)	Free tiers sufficient for personal use
Container	Docker Compose	Local dev stack: n8n + Redis + Ollama + Supabase
Monitoring	OpenTelemetry + Axiom	Traces, logs, latency tracking for AI calls

6.1 Infrastructure Cost Model

Conductor OS is designed to run at near-zero cost for personal use. The following estimates assume a single operator with 10–15 connected sources and moderate daily usage (2–4 active hours):

- Vercel Hobby (frontend): $0/month
- Supabase Free (database + auth + realtime): $0/month
- Upstash Redis (job queue): $0/month for under 10,000 commands/day
- HuggingFace Inference API: $0/month (free tier sufficient)
- Claude API (synthesis, digest, Q&A): $3–8/month for typical personal usage
- Railway/Render (n8n + backend): $5/month
- Ollama (local LLM, optional): $0/month — compute on operator's machine

> *Total estimated monthly cost for full personal deployment: $8–13 USD, of which $3–8 is LLM API usage. The system functions with reduced AI capability at zero marginal cost.*

7. Integration Catalog

Each integration is implemented as a standardized adapter that conforms to the OmniSignal schema. The adapter pattern means new integrations can be added without modifying the core pipeline. The following catalog represents the v1.0 integration set.

7.1 Communication Platforms

Platform	Method	Bidirectional
Telegram	Bot API + webhooks	Receive messages; send messages, media, inline keyboards
Discord	Gateway API + bot	Receive from guilds/channels; send messages, embeds
Slack	Events API + webhooks	Receive from workspaces; send to channels
Email (IMAP/SMTP)	NodeMailer + IMAP	Receive via IMAP polling; send via SMTP
WhatsApp	Meta Cloud API	Receive messages; send text and templates

7.2 Developer and Technical Platforms

Platform	Method	Bidirectional
GitHub	REST API v3 + webhooks	Receive: issues, PRs, commits, stars; Act: create issues, comment, merge
GitLab	REST API + webhooks	Receive: MRs, pipelines; Act: create MRs, trigger CI
Linear	GraphQL API	Receive: issue updates; Act: create and update issues
Jira	REST API v3	Receive: ticket updates; Act: create tickets, add comments
Vercel	REST API	Receive: deployment events; Act: trigger deployments

7.3 Content and Information Sources

Source	Method	Notes
RSS / Atom feeds	Polling + FeedParser	Any public feed; configurable poll interval
YouTube	Data API v3	Channel videos, comments, live chat; quota-aware
Twitter / X	Filtered Stream v2	Real-time keyword and account monitoring
Reddit	OAuth2 API	Subreddit monitoring, comment threads
HackerNews	Algolia Search API	Real-time story and comment monitoring
arXiv / Semantic Scholar	REST API	Academic paper monitoring by category or keyword
GitHub Trending	Web scrape via Puppeteer	Daily trending repositories by language
Substack	RSS	Publication monitoring via RSS adapter

7.4 Financial and Blockchain Sources

Source	Method	Notes
CoinGecko	REST API (free)	Price, volume, market cap; configurable alert thresholds
Glassnode	REST API	On-chain analytics: UTXO cohorts, SOPR, whale movements
Etherscan	REST API	Wallet monitoring, contract events, gas tracker
DeFiLlama	REST API (free)	TVL tracking across protocols and chains
Binance / OKX	WebSocket streams	Real-time order book, trade, and funding rate data

7.5 Creative and Production Platforms

Platform	Method	Bidirectional
DistroKid / RouteNote	REST API	Receive: streaming stats, royalty reports
FilmFreeway	RSS + web	Receive: festival deadlines, submission status

Platform	Method	Bidirectional
Vimeo	REST API	Receive: view analytics; Act: upload, manage videos
SoundCloud	REST API	Receive: play counts, comments; Act: post tracks

8. Data Architecture

8.1 PostgreSQL Schema (Supabase)

```
-- Core signal store
CREATE TABLE signals (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  operator_id   UUID REFERENCES operators(id),
  source        TEXT NOT NULL,
  source_type   TEXT NOT NULL,
  content       TEXT NOT NULL,
  title         TEXT,
  url           TEXT,
  author        TEXT,
  timestamp     TIMESTAMPTZ NOT NULL,
  relevance     FLOAT,
  sentiment     FLOAT,
  topics        TEXT[],
  entities      TEXT[],
  embedding     VECTOR(384),   -- pgvector: all-MiniLM-L6-v2
  is_wildcard   BOOLEAN DEFAULT FALSE,
  raw           JSONB,
  created_at    TIMESTAMPTZ DEFAULT now()
);

-- Operator persona
CREATE TABLE operators (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name          TEXT NOT NULL,
  role          TEXT,
  location      TEXT,
  interests     TEXT[],
  active_goal   TEXT DEFAULT 'monitoring',
  entropy_level INT  DEFAULT 10,
  sources       JSONB,
  models        JSONB,
  created_at    TIMESTAMPTZ DEFAULT now()
);

-- Semantic search index
CREATE INDEX signals_embedding_idx ON signals
  USING hnsw (embedding vector_cosine_ops);
```

8.2 Redis Data Model (BullMQ)

Redis is used exclusively for the enrichment job queue and real-time signal deduplication. It is not used for persistent storage. Job queues are structured by priority: high-relevance signals enter the priority queue and are enriched within 500ms; low-relevance signals enter the standard queue and are enriched within 60 seconds.

9. Intention Engine — The Core Differentiator

The Intention Engine is what transforms Conductor OS from an advanced aggregator into a true cognitive prosthetic. It maintains a continuous model of the operator's current task state and uses it to reshape every aspect of the system's behavior.

9.1 Goal Modes

Five primary goal modes are built in, each with a different weight matrix and default behaviors:

- RESEARCH — Maximizes depth and source diversity. Prioritizes academic papers, technical documentation, long-form analysis. Increases wildcard ratio to 20%. Enables auto-summarization of long signals.
- MONITORING — Maximizes recency and alert sensitivity. Prioritizes real-time streams, sentiment shifts, entity mentions. Enables push notifications for configured trigger conditions.
- CREATIVE — Maximizes serendipity and cross-domain connection. Wildcard ratio increases to 30%. Deprioritizes operational and financial signals. Enables AI connection-mapping mode.
- EXECUTION — Minimizes distraction, maximizes action affordances. Shows only high-priority signals requiring a response. Every signal card shows a "next action" button. Suppresses wildcard channel.
- COMMUNICATION — Prioritizes incoming messages across all messaging platforms. Aggregates Telegram, Discord, Slack, email into a unified queue ranked by urgency and relationship strength.

9.2 Intention Vector Computation

```
function computeIntentionVector(operator: Operator): Float32Array {
  const base = operator.interests.map(i => interestWeight(i));
  const goalBoost = GOAL_WEIGHT_MATRIX[operator.activeGoal];
  const temporal = recentSignals(operator.id, 24h).topicFrequency();

  // Weighted combination: base interests + goal mode + recent activity
  return normalize(
    base.map((w, i) => w * 0.4 + goalBoost[i] * 0.4 + temporal[i] * 0.2)
  );
}
```

9.3 Operator Context Injection

When the Intention Engine dispatches a signal to the LLM Gateway, it constructs a structured context object that captures the full operator state at the moment of the request. This context is injected into every LLM call, ensuring that responses are grounded in the operator's actual situation rather than a generic interaction.

10. Security and Privacy Model

10.1 Data Sovereignty Tiers

Conductor OS offers three deployment models with different data sovereignty characteristics:

- Cloud-hosted (default): Supabase-managed database with end-to-end encryption at rest. Signal content is never shared with third parties. HuggingFace and Claude API calls transmit signal content to external services.
- Self-hosted cloud: Operator runs their own Supabase instance and n8n on a private VPS. All data remains under operator control. HuggingFace and Claude API calls still transmit content externally.
- Full local (privacy mode): Ollama replaces all external AI APIs. Zero data leaves the operator's machine. Supabase can be replaced with a local PostgreSQL instance. This mode is designed for operators with strict data sovereignty requirements.

10.2 Authentication and Authorization

- All API routes are protected by JWT verification via Supabase Auth
- Row Level Security (RLS) policies enforce that operators can only access their own signals, persona configuration, and action history
- Webhook endpoints are validated with HMAC-SHA256 signatures for all platforms that support it
- n8n credentials are encrypted at rest using AES-256
- API keys for third-party services are stored in environment variables, never in the database

10.3 API Credential Management

External service credentials are managed through a dedicated secrets interface. The system supports environment variable injection (recommended for self-hosted), Supabase Vault for encrypted per-operator credentials, and integration with secret managers such as HashiCorp Vault and AWS Secrets Manager for team deployments.

11. Deployment Architecture

11.1 Local Development Stack (Docker Compose)

```
# docker-compose.yml
services:
  postgres:
    image: supabase/postgres:15
    environment:
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes: [./data/postgres:/var/lib/postgresql/data]

  redis:
    image: redis:7-alpine
    volumes: [./data/redis:/data]

  n8n:
    image: n8nio/n8n:latest
    environment:
      DB_TYPE: postgresdb
      DB_POSTGRESDB_HOST: postgres
    ports: ["5678:5678"]
    volumes: [./data/n8n:/home/node/.n8n]

  ollama:
    image: ollama/ollama:latest
    volumes: [./data/ollama:/root/.ollama]
    ports: ["11434:11434"]
    deploy:
      resources: { reservations: { devices: [{ capabilities: [gpu] }] } }

  conductor-api:
    build: ./apps/api
    environment:
      DATABASE_URL:
postgresql://postgres:${POSTGRES_PASSWORD}@postgres:5432/conductor
      REDIS_URL: redis://redis:6379
      ANTHROPIC_API_KEY: ${ANTHROPIC_API_KEY}
      OLLAMA_BASE_URL: http://ollama:11434
```

11.2 Production Deployment

The recommended production topology for a personal or small-team deployment:

- Frontend (Next.js): Vercel — free hobby tier supports all personal use cases. Edge runtime for low-latency global delivery.
- API server (Node.js + tRPC): Railway or Render — $5/month starter tier. Includes automatic SSL, health checks, and zero-downtime deploys.

- Database: Supabase Cloud — free tier includes 500MB storage, 2GB bandwidth, and all needed features for personal use.
- Job queue (Redis): Upstash — free tier includes 10,000 commands/day, sufficient for personal use.
- n8n: Self-hosted on Railway or a $6/month Hetzner VPS — n8n Cloud is also available at $20/month.
- Ollama: Local machine — MacBook M-series or Linux with NVIDIA GPU recommended.

12. Implementation Roadmap

Phase	Timeline	Deliverables
Phase 0	Weeks 1–2	Repository scaffold, Docker Compose dev stack, Supabase schema, tRPC skeleton, Next.js shell with auth
Phase 1	Weeks 3–5	Layer 0: RSS, GitHub, Telegram adapters. Layer 1: LDA + TF-IDF local processing. Basic feed UI with relevance scoring
Phase 2	Weeks 6–8	Layer 2: HuggingFace enrichment queue via BullMQ. Layer 3: Adaptive filter with goal modes. Persona setup interface
Phase 3	Weeks 9–11	Layer 4: n8n action dispatch. Bidirectional actions: Notion, Telegram, GitHub. Action history and audit log
Phase 4	Weeks 12–14	Layer 5: Claude API gateway with persona injection. Digest generation, Q&A, connection mapping. Ollama integration
Phase 5	Weeks 15–17	Extended integration catalog: Discord, Slack, financial APIs, creative platforms. Webhook ingestion for all platforms
Phase 6	Weeks 18–20	pgvector semantic search. Attention map visualization. Cross-signal entity graph. Temporal timeline view
Phase 7	Weeks 21–24	Mobile PWA with push notifications. Offline mode with local-first sync. Public release and documentation

13. Use Cases

13.1 Independent Researcher / Polymath

An independent researcher working across cryptography, AI systems, and philosophy of mind receives 200–400 signals per day from arXiv, HuggingFace Papers, academic Twitter, and specialist Telegram channels. The problem is not access to information — it is synthesis. Without a system that understands their cross-domain research program, every signal must be evaluated manually against a mental model that is never fully externalized.

Conductor OS deployment: RESEARCH mode with a 20% wildcard ratio. The Intention Engine is configured with a weighted interest graph that links cryptography, information theory, and consciousness research. Every morning, the system generates a digest that identifies connections across the previous 24 hours of signals — pointing to the paper on Kolmogorov complexity that connects to the DeFi architecture they are designing, or the philosophy of attention paper that maps onto their LLM fine-tuning approach.

13.2 Technical Entrepreneur / Systems Architect

A technical founder managing multiple simultaneous projects — a DAO platform, a filmmaking tool, and a cryptographic puzzle system — faces the coordination problem at scale. Relevant signals are scattered across GitHub repositories, Telegram groups, financial APIs, n8n automation logs, and customer communication channels. No single tool surfaces all of this in a unified, intention-aware interface.

Conductor OS deployment: A project-switching Intention Engine configuration. When the operator sets their current project context, the relevance weights, source priorities, and action affordances reconfigure accordingly. A GitHub webhook event about the DAO platform appears at the top of the feed when in DAO mode; the same event is suppressed in FILM mode. The bidirectional action layer allows the operator to create Linear tickets, send Telegram announcements, and trigger n8n deployment workflows directly from the feed card, without leaving the interface.

13.3 Independent Creator / Filmmaker

An independent filmmaker using AI tools for production monitoring needs to track developments across Runway, Pika, ElevenLabs, and Sora simultaneously, while also monitoring festival submission deadlines on FilmFreeway and distributor announcements. The creative research

dimension requires exposure to adjacent ideas — philosophy, architecture, music theory — that might inform the next project.

Conductor OS deployment: CREATIVE mode with a high wildcard ratio (25–30%). The system integrates FilmFreeway RSS, YouTube channel monitoring for specific directors, and the DistroKid API for music distribution analytics. The AI synthesis layer is configured to identify thematic connections between the operator's recent film projects and emerging ideas in the feed. The action layer connects directly to Notion for script notes and to Telegram for sharing discoveries with collaborators.

13.4 DeFi / Crypto Active Participant

An active participant in decentralized finance requires real-time monitoring of on-chain events, protocol governance votes, market microstructure, and research publications — simultaneously. The latency requirements are strict (whale wallet movements are actionable within minutes), but the synthesis requirements are also high (understanding why a protocol's TVL is moving requires reading multiple sources and identifying the causal chain).

Conductor OS deployment: MONITORING mode with configured alert thresholds. Etherscan webhooks fire for tracked wallet addresses; Glassnode polling surfaces UTXO cohort shifts; DeFiLlama tracks TVL movements across protocols. Alert conditions are defined in n8n workflows and surface as priority-0 signals in the feed. When a complex on-chain event triggers multiple alerts, the operator can invoke the LLM synthesis layer with a single gesture to generate a structured analysis of the causal chain across all the related signals.

14. Open-Source Model and Community

Conductor OS is designed from the ground up to be open-source, self-hostable, and community-extensible. The core architecture — signal adapters, processing layers, Intention Engine, and action dispatch — is released under the MIT license. The adapter registry is the primary extension point: any developer can contribute a new integration by implementing the OmniAdapter interface and submitting to the community registry.

14.1 Extension Points

- Signal adapters — implement OmniAdapter to add any data source
- Action handlers — implement ActionHandler to add new dispatch targets
- Goal modes — define custom weight matrices and behavior sets for domain-specific use cases
- Micro-models — swap any Tier 1 or Tier 2 model for domain-specific alternatives
- UI widgets — the dashboard is composed of independently mountable React components; operators can add, remove, and rearrange widgets

14.2 Community Registry

The Conductor OS community registry will catalog community-contributed adapters, goal mode presets, and n8n workflow templates. The registry is moderated for security (all adapters are reviewed for credential handling and data transmission practices) and indexed for discoverability by integration category, platform, and operator use case.

> *The long-term vision for Conductor OS is an ecosystem of operator profiles — sharable persona configurations that encode an operator's interest graph, source set, goal modes, and workflow templates. A researcher can share their RESEARCH profile; a filmmaker can share their PRODUCTION profile. The community builds the collective intelligence layer.*

15. Appendix — API Reference

15.1 OmniAdapter Interface

```
interface OmniAdapter {
  id:          string
  name:        string
  type:        "POLL" | "WEBHOOK" | "STREAM" | "PUSH"
  pollInterval?: number          // ms, only for POLL type

  // Initialize connection, validate credentials
  connect(config: AdapterConfig): Promise<void>

  // For POLL adapters: fetch new signals since lastSeen
  fetch(lastSeen: Date): Promise<OmniSignal[]>

  // For WEBHOOK adapters: validate and parse incoming payload
  parseWebhook(payload: unknown, headers: Headers): OmniSignal[]

  // For STREAM adapters: return an async generator
  stream?(): AsyncGenerator<OmniSignal>

  // Bidirectional: send an action through this adapter
  dispatch?(action: AdapterAction): Promise<ActionResult>

  // Health check
  ping(): Promise<{ ok: boolean; latencyMs: number }>
}
```

15.2 ActionHandler Interface

```
interface ActionHandler {
  id:       string
  name:     string
  triggers: ActionTrigger[]  // signal conditions that surface this action

  // Render action button label given a signal
  label(signal: OmniSignal): string

  // Execute the action
  execute(signal: OmniSignal, params: ActionParams): Promise<ActionResult>

  // Optional: define an n8n workflow to trigger instead
  n8nWorkflowId?: string
}
```

15.3 Environment Variables

Variable	Description
ANTHROPIC_API_KEY	Claude API key — required for LLM synthesis layer
SUPABASE_URL	Supabase project URL
SUPABASE_SERVICE_KEY	Supabase service role key (server-side only)
SUPABASE_ANON_KEY	Supabase anonymous key (client-side)
REDIS_URL	Redis connection string for BullMQ job queue
N8N_BASE_URL	n8n instance base URL for workflow dispatch
N8N_API_KEY	n8n API key for workflow triggers
OLLAMA_BASE_URL	Ollama instance URL (default: http://localhost:11434)
HF_API_TOKEN	HuggingFace API token — optional, increases rate limits
TELEGRAM_BOT_TOKEN	Telegram Bot API token
GITHUB_APP_ID	GitHub App ID for webhook integration
GITHUB_PRIVATE_KEY	GitHub App private key (PEM format)
NEXT_PUBLIC_APP_URL	Public URL of the deployed application

Conductor OS — Personal Intelligence Operating System

White Paper v1.0 · March 2026 · Open Source under MIT License

You are the conductor. Every service in your digital life is an instrument.